

# AI活用(6) ソフトウェア開発(2)

テストと品質

情報科学専攻 | 松野 裕

2025年7月8日 (対面)

# 本日のアジェンダ

---

1. テストの基礎 (10分)
2. AIによるテストコード生成 (15分)
3. AIによるコードレビュー (15分)
4. AIによるリファクタリング (10分)
5. ドキュメント自動生成 (5分)
6. 演習 (45分)

# テストの基礎

なぜテストが重要なのか

# なぜテストが重要か

---

## ソフトウェアテストの目的

- **バグの早期発見** — 修正コストは後工程ほど高くなる
- **品質保証** — 期待通りに動作することを保証
- **研究の再現性確保** — 実験コードが正しく動くことを保証

## 研究コードにおけるテストの重要性

- 実験結果の信頼性に直結
- 論文の査読で再現性が問われる / コード修正時の回帰確認

テストのないコードは「たまたま動いている」だけかもしれない

# ソフトウェアテストの基礎

| テストの種類 | 対象         | 目的              |
|--------|------------|-----------------|
| 単体テスト  | 個々の関数・メソッド | 各部品が正しく動くか      |
| 結合テスト  | モジュール間の連携  | 部品を組み合わせて正しく動くか |
| 回帰テスト  | 変更後のシステム全体 | 修正で他の部分が壊れていないか |

## テストケースの種類

- **正常系**: 期待通りの入力での動作確認
- **異常系**: 不正な入力でのエラーハンドリング確認
- **境界値**: 限界値付近での動作確認（0、空文字、最大値など）

# AIによるテストコード生成

自動生成ワークフローとプロンプト設計

# テストコード自動生成のワークフロー

---

## Step 1: テスト対象のコードをAIに渡す

以下のPython関数に対するpytestのテストケースを作成してください。 正常系3つ、異常系2つ、境界値2つ。各テストにコメントで説明をつけてください。 [テスト対象のコード] 

```
def calculate_average(numbers: list[float]) -> float: ...
```

## Step 2: 生成されたテストを実行

## Step 3: 不足しているケースを追加依頼

# 効果的なテストプロンプトのポイント

---

## 明示すべき項目

1. テストフレームワーク — pytest、unittest など
2. テストの種類と数 — 正常系〇個、異常系〇個
3. カバレッジの要求 — 分岐網羅、条件網羅
4. エッジケースの考慮 — 空リスト、None、負数など

## 追加で依頼できること

- テストのパラメータ化 (`@pytest.mark.parametrize`)
- fixtureの作成
- モック (mock) の使用
- カバレッジレポートの確認方法



# AIによるコードレビュー

活用法とチェックリスト

# AIコードレビューの活用法

---

## プロンプト例

以下のPythonコードをレビューし、バグ・パフォーマンス・可読性・セキュリティの観点で問題を指摘してください。各指摘には重要度（高/中/低）と修正案を含めてください。 [コードを貼り付け]

## AIレビューの活用シーン

- 提出前の最終チェック
- 自分では気づきにくい問題の発見
- コーディング規約への準拠確認

# コードレビューチェックリスト

| 観点     | チェック項目                   |
|--------|--------------------------|
| 可読性    | 変数名は適切か、コメントは十分か、関数は短いかな |
| 効率性    | 不要なループはないか、適切なデータ構造かな    |
| セキュリティ | 入力値の検証、SQLインジェクション対策     |
| エラー処理  | 例外処理は適切か、エラーメッセージは明確かな   |
| 再利用性   | 関数の汎用性、モジュール化            |
| テスト容易性 | テストしやすい設計か、依存関係は少ないかな    |

AIにこのチェックリストを渡してレビューを依頼すると効果的

# AIによるリファクタリング

種類と適用場面

# リファクタリングの種類と適用場面

---

## プロンプト例

このコードをより保守しやすくリファクタリングしてください。 変更箇所とその理由も説明してください。 [コードを貼り付け]

# よくあるリファクタリング

| 種類       | 内容                | 適用場面             |
|----------|-------------------|------------------|
| 関数分割     | 長い関数を複数の短い関数に     | 50行以上の関数         |
| 命名改善     | より分かりやすい変数名・関数名に  | x , tmp などの曖昧な名前 |
| 設計パターン適用 | Strategy、Factory等 | 条件分岐が多い場合        |
| 重複排除     | DRY原則の適用          | 同じ処理が複数箇所にある     |

# ドキュメント自動生成

docstring・README の自動作成

# AIによるドキュメント生成

## プロンプト例

以下のPythonコードに対して、各関数のdocstring (Google style)、モジュール全体のREADME (使い方、依存関係、実行方法)、使用例を生成してください。 [コードを貼り付け]

## ドキュメントの重要性

- 研究コードの **再現性** を確保
- 共同研究者との **情報共有** / 将来の自分のための **記録**

「3ヶ月前の自分は他人」 — ドキュメントがないと自分のコードも読めなくなる



# 演習

テスト・レビュー・リファクタリング

## AI演習 演習1（15分）：テストケース生成

---

### 前回作成したコードにテストを追加

1. 前回のPythonスクリプトをGemini Proに渡す
2. pytestのテストケースを生成させる
  - 正常系3つ、異常系2つ、境界値2つ
3. 生成されたテストを **実行** して確認
4. 不足しているテストケースがあれば追加依頼

### ポイント

- テストが全部パスすることを確認
- テストの内容を理解する（AIの説明を読む）

## AI演習 演習2（15分）：AIにコードレビューさせ改善

### 手順

1. 前回のコード + テストコードをAIに渡す
2. レビューチェックリストを添えてレビュー依頼
3. 指摘された問題を **重要度順に修正**
4. 修正後、テストが通ることを確認

### レビュー依頼テンプレート

以下のコードをレビューしてください。観点：可読性、効率性、エラー処理、セキュリティ。各指摘に重要度（高/中/低）と修正案をつけてください。

## AI演習 演習3（15分）：リファクタリングとドキュメント

---

### 手順

1. レビュー後のコードをAIにリファクタリング依頼
2. 変更点と理由を確認
3. docstringを生成させる
4. 簡単なREADME（使い方）を生成させる

### 最終成果物

- テスト付きのPythonコード
- docstring付き
- 簡単な使い方ドキュメント

# 研究コードの品質管理

---

## 再現性の確保

- **requirements.txt** — 依存パッケージとバージョンを記録
- **README** — 実行手順を明記
- **テスト** — 動作を保証

## バージョン管理の重要性

- **Git** でコードの変更履歴を管理
- 実験ごとにブランチやタグを作成
- コミットメッセージで変更内容を記録

研究コードも「ソフトウェア」 — **品質管理の基本を守る**

# 課題

---

## 提出物

1. テスト付きのPythonコード（前回コード + テスト、全パス確認済み）
2. AIコードレビュー結果レポート
  - AIが指摘した問題点と各問題への対応
  - リファクタリング前後の比較

**提出期限:** 次回授業（7/15）まで

# 次回予告

---

## 第13回（最終回）：成果発表とピアレビュー

- 研究ポートフォリオの成果発表
- ピアレビュー
- AI活用の振り返り

## 準備

- 発表スライド（5枚程度）を準備
- これまでの成果物を整理しておく